Akademia Nauk Stosowanych w Nowym Sączu
Wydział Nauk Inżynieryjnych • Informatyka • Studia I stopnia

Niezawodność systemów informatycznych

Zajęcia projektowe nr 2, blok P2

Testy strukturalne: pokrycie kodu

1 Informacje organizacyjne

- **Blok tematyczny z karty zajęć:** P2: Testy strukturalne
- **Forma pracy:** ten sam zespół i ten sam wariant SUT co na zajęciach nr 1

Cel dzisiejszych zajęć

Po dzisiejszych zajęciach każdy zespół powinien:

1. rozumieć różnicę między pokryciem instrukcji, gałęzi i warunków,
2. mieć zainstalowane i uruchomione narzędzie do pomiaru pokrycia kodu,
3. mieć zestaw testów jednostkowych, który świadomie dąży do wysokiego pokrycia wybranych metod SUT,
4. umieć przeczytać raport pokrycia i wskazać w nim niepokryte fragmenty.

Jak to się ma do zajęć nr 1

Na zajęciach nr 1 testowaliście SUT eksploracyjnie, czyli bez planu, różnymi danymi wejściowymi. Dziś robicie coś przeciwnego: testy strukturalne są planowane na podstawie kodu źródłowego, tak żeby celowo wykonać każdą instrukcję i każdą gałąź warunku. To dwie różne strategie wykrywania usterek, uzupełniające się nawzajem.

2 Wprowadzenie teoretyczne

2.1 Testy białoskrzynkowe a czarnoskrzynkowe

Testy strukturalne (białoskrzynkowe)

Testy projektowane na podstawie struktury kodu źródłowego (instrukcji, gałęzi, warunków), a nie wyłącznie na podstawie specyfikacji. Osoba piszącą test zna implementację i świadomie dobiera dane wejściowe tak, aby wykonać konkretne fragmenty kodu.

2.2 Poziomy pokrycia kodu

Pokrycie instrukcji (statement coverage)

Odsetek instrukcji w kodzie, które zostały wykonane przynajmniej raz podczas testów. Najsłabsza forma pokrycia: nie sprawdza, czy obie gałęzie warunku zostały przetestowane.

Pokrycie gałęzi (branch coverage)

Odsetek gałęzi instrukcji warunkowych (zarówno prawda, jak i fałsz), które zostały wykonane podczas testów. Silniejsze niż pokrycie instrukcji: wymusza przetestowanie obu stron każdego `if`.

Pokrycie warunków i MC/DC

Pokrycie warunków wymaga, żeby każdy pojedynczy warunek logiczny w złożonym wyrażeniu (np. `a and b`) przyjął zarówno wartość prawda, jak i fałsz. MC/DC (*Modified Condition/Decision Coverage*) idzie o krok dalej: wymaga, żeby każdy warunek niezależnie wpływał na wynik całego wyrażenia. To bardzo rygorystyczny standard, wymagany np. w oprogramowaniu lotniczym (norma DO-178C).

100% pokrycia to nie to samo co brak usterek

Pokrycie mówi wyłącznie, który kod został *wykonany*, a nie czy wynik tego wykonania został *sprawdzony*. Test, który wywołuje funkcję i nic nie asercjonuje, może dawać 100% pokrycia instrukcji i nie wykryć żadnej usterki. Pokrycie jest miarą kompletności testów, nie miarą ich skuteczności.

Dlaczego 100% pokrycia bywa mylące

Dwa zespoły mogą osiągnąć identyczne 100% pokrycia gałęzi tego samego kodu, a mieć zupełnie różną szansę wykrycia usterek, w zależności od tego, jak precyzyjne są ich asercje i jakie wartości brzegowe wybrały. Dlatego pokrycie kodu traktuje się jako warunek konieczny, ale niewystarczający dobrych testów, dokładność sprawdzenia dopełniają testy mutacyjne, którymi zajmiemy się na kolejnych zajęciach.

2.3 Narzędzia

coverage.py i pytest-cov

`coverage.py` to standardowe narzędzie do pomiaru pokrycia kodu w Pythonie. `pytest-cov` to wtyczka integrująca je z `pytest`, tak żeby raport pokrycia powstawał automatycznie przy uruchamianiu testów.

Uruchomienie testów z pomiarem pokrycia

```
pip install pytest pytest-cov
pytest --cov=zespol_A --cov-report=term-missing --cov-report=html
```

Flaga `term-missing` pokazuje w terminalu numery niepokrytych linii, a `html` generuje szczegółowy raport w folderze `htmlcov/`, który można otworzyć w przeglądarce.

3 Zadanie projektowe na dziś

1. **Zainstalujcie narzędzia** `pytest`, `pytest-cov` (ewentualnie odpowiedniki dla innego języka, jeśli pracujecie poza Pythonem).

2. **Wybierzcie co najmniej trzy metody** swojego wariantu SUT (np. `wplac`, `wyplac`, `przelew`) i napiszcie dla nich testy jednostkowe pokrywające możliwie wszystkie gałęzie, w tym przypadki wyjątkowe (konto zablokowane, kwota niedodatnia, niewystarczające środki, przekroczony limit).

3. **Zmierzcie pokrycie** wybranych metod za pomocą `coverage.py` / `pytest-cov` i zapiszcie wynik procentowy.
4. **Przejrzyjcie raport** i zidentyfikujcie niepokryte linie lub gałęzie.
5. **Dopiszcie dodatkowe testy**, które domykają braki znalezione w poprzednim kroku, i zmierzcie pokrycie ponownie.

Częsty błąd

Pisanie testu, który wywołuje metodę, ale nie sprawdza wyniku żadną asercją (`assert`). Taki test podnosi pokrycie, ale nie wykryje żadnej usterki, bo nic nie porównuje z oczekiwaną wartością.

Co trafia do raportu

Na koniec dzisiejszych zajęć zespół dopisuje do wspólnego raportu projektowego:

- listę przetestowanych metod wraz z liczbą napisanych przypadków testowych,
- wynik pokrycia przed i po uzupełnieniu testów (instrukcje, gałęzie),
- krótki opis niepokrytych fragmentów, które świadomie pominięto, i dlaczego,
- wnioski: czy w trakcie pisania testów strukturalnych znaleziono zachowanie niezgodne z oczekiwaniami.

4 Pytania kontrolne

1. Czym różni się pokrycie instrukcji od pokrycia gałęzi? Podaj przykład kodu, dla którego oba poziomy dają różny wynik.
2. Dlaczego 100% pokrycia instrukcji nie gwarantuje wykrycia wszystkich usterek?
3. Co to jest MC/DC i w jakim kontekście stosuje się ten standard?
4. Jaka jest różnica między testem, który podnosi pokrycie, a testem, który rzeczywiście weryfikuje poprawność działania kodu?

5 Przygotowanie do zajęć nr 3 (blok P2, ciąg dalszy)

Zajęcia nr 3 kontynuują blok P2 i dotyczą testów mutacyjnych, czyli sprawdzenia, jak dobre są testy napisane dziś. Na następne zajęcia:

- zachować napisane dziś testy jednostkowe w repozytorium, będą punktem wyjścia,
- mieć działające środowisko z zajęć nr 2 (pytest, coverage).